Problem Solving using Computer

- Define the problem
- Analyze the solutions
- Develop the Algorithm
- Write the pseudocode or draw the flowchart
- Coding using a programming language
- Debugging and Execution
- Verification and Testing

Define the problem

- State clearly what is given and what is required to be achieved
- Example problem: Compute the sum of squares of two numbers
- Definition of the above problem: Given any two numbers, say N1, N2 obtain the output $N1^2 + N2^2$

Analyze the solutions

- Find out the different methods to solve the problem i.e., to find out the solutions and decide the best method
- For the example problem, the methods can be as follows

  Method1: Solution = N1 * N1 + N2*N2

  Method2: Solution = Square(N1) + Square(N2)

  where square is a built-in function

Develop the Algorithm

- Algorithm is a method to find the desired solution

//Algorithm for sum of squares of two numbers

- Input two numbers
- Compute square of each number
- Compute the sum of value of each square
- Display the output

Pseudocode

- Pseudocode is similar to the programming code in form except for the syntax of the program
- Pseudocode for the sum of squares problem
- Read N1, N2
- Sum1 = N1* N1
- Sum2 = N2* N2
- Sum = Sum1 + Sum2
- Print Sum

Difference between Algorithm and Pseudocode

- An Algorithm is a well defined sequence of steps that provides a solution to the given problem
- A pseudocode is one of the methods to represent an algorithm

Coding

```
 /* C program to find sum of squares*/
#include <stdio.h>
main()
{
   float n1, n2;
printf("Please enter two numbers\n");
scanf("%d  %d \n", &n1, &n2);
 sum1= n1*n1;
 sum2 = n2*n2;
 sum = sum1 + sum2;
 printf ("The result is  ", sum);
}
```

Debugging

- The process of identifying and removing errors or correcting errors in a program is called debugging
- The errors can be syntactic errors or semantic errors
- Syntactic errors: These errors occur due to violation of rules of programming language

  for example missing semicolon or missing

  parenthesis
- These are identified during compilation

Semantic errors

- Semantic errors or logical errors occur when you run the program after compilation errors are corrected
- These will not produce error messages but the desired results are not obtained due to wrong understanding of problem or due to wrong code
- For example, N = N1 * N2 is coded whereas you wanted N = N1/N2
- Divide by 0 gives semantic error

Verification and Testing

- Let us assume debugging and execution is successful
- Please enter two numbers

3.0 4.0

The result is 25.0

Testing with different data sets and checking whether desired results are obtained or not is called verification

Exercises

Write algorithms for the following problems

1. Display largest of two numbers
2. Display largest of three numbers
3. Convert temperature from Fahrenheit to Centigrade
4. Convert input in feet to output in centimeters
5. Compute GCD (greatest common devisor) of two integers

Display largest of two numbers

```
// Algorithm to find largest of two numbers
Read X, Y
 if  X > Y then Large = X
 if Y > X Large = Y
Print Large
```

or

```
Read X, Y
 if  X > Y then Large = X
 else Large = Y
 Print Large
```

Display largest of three numbers

```
//Algorithm to find largest of three different
//numbers
  Read X, Y, Z
 If (X > Y) and (X > Z) then Large = X
 If (Y > X) and (Y > Z) then Large = Y
 If (Z > X) and (Z > Y) then Large = Z
 Print Large
```

Another Algorithm for Largest of three numbers

read A, B, C

Large = A

if B > Large then Large = B

if C > Large then Large = C

print Large

Convert temperature from Fahrenheit to Centigrade

// Algorithm to Convert temperature from
// Fahrenheit to Centigrade
Read F
C = (5/9) (F -32)
Print C

Convert input in feet to output in centimeters

```
// Convert feet to centimeters
 Read feet
// One feet = 12 inches
// One inch = 2.5 centimeters
FIC = 12 * 2.5
Print FIC
```

Compute GCD (greatest common devisor) of two integers

```
  //Compute GCD (greatest common devisor) of two integers
// GCD with iteration
   //using while do
  read a, b  // a > b > 0

   while b < > 0  do
   t = b; b = a mod b; a = t
   repeat
   print a

   //GCD using if then else//
    read a, b  // a > b > 0
  L1: if b=0 then  print a
 else t = b; b = a mod b; a = t;
 go to L1
 endif
```

Representing Programming concepts in Algorithms

- Algorithms Play an important role in software development
- Algorithm is a special method (usable by computer) of solving a problem
- the method is composed of finite set of steps, each step requiring some operation to be performed

Characteristics of algorithms

- Each operation must be definite, i.e. it must be clear what should be done e.g.compute 125/5, add 60 to 70

  Example of not definite or ambiguous operations 5/0, add 2 or 3 to x.
- Each operation should be effective i.e. it can be performed in a finite amount of time Arithmetic on integers is effective but not arithmetic on real numbers.
- Algorithm must terminate after a finite number of operations (while loops which never terminate should not appear in an algorithm)
- Operating System (OS) is an exception as OS never terminates

Expressing Algorithms

- Structured programming concepts as sequence, selection and repetition and recursion are used to express algorithms

- Primitive data types - Integer, Real, Boolean, Character
- Boolean variables : true and false represented as 1 and 0

  integer x,y; boolean a, b; char c, d
- assignment operation

  variable = (expression)
- Logical operators: and , or , not
- Relational operators: <, <=, >, >=, < >

Arrays

A number of variables of the same type are represented using arrays

Array declaration: integer A (l_1:u_1,..l_n:u_n)

Example: integer A(10)

Lower bound is assumed to be 1

Matrices are two dimensional arrays

Mat(3,3)

Matrix (10, 15)

Selection

- Selection out of two alternatives
- if cond then S1 endif
- if cond then S1
  else S2
  endif
- Selection with many alternatives - case

```
case
        :cond1   :S1
        :cond 2  :S2
         .   .   .
        :cond n  :Sn

        :otherwise  : Sn+1
 endcase
```

Si 1<=i <=n+1 are group of statements

Iteration statements

1. While cond do
   S
   Repeat
2. Loop
   S
   Until cond repeat
3. For variable ← start to finish by increment do
   S
   Repeat (variables start, finish, increment are arithmetic expressions)

Iteration statements contd..

4.(a) loop

S

repeat

(b) loop

S1

if cond then exit endif

S2

repeat

(c) loop

S1

if cond1 then cycle endif

S2

until cond2

repeat

Cycle causes a transfer of control to the last statement of the iterative statement which contains it

- goto label L

control goes to the statement with label L where L is a number or a name

Functions and Procedures

- There are built-in functions (library functions) and user defined functions

- Difference between function and procedure is function always returns a value whereas procedure may or may not

- Functions are used when the value they return are used in an expression as
- if Prime (n) and equal (m,n) then go to 60

- In algorithms we use the keyword procedure for function also

Procedures contd..

procedure name (parameter list)
// parameter list contains formal parameters
declarations
S
end name
S represents programming statements

return (expr) to return the value expr by a function
// this is a comment
stop – to halt execution

Calling Procedures

- There are two ways of calling procedures –
  call by value and call by reference
- Actual parameters or arguments are used when procedures are called
- In call by value actual values are passed
- In call by reference address of values are passed
- At execution time formal parameters are replaced by actual parameters

Global and Local variables

- Global variables are declared outside the procedure or in a procedure which contains the current procedure
- Local variables are declared inside a procedure
- In algorithms only global variables are declared

Example: procedure to compute the maximum of an array

```
    Procedure MAX (A,n,j)
  //set j so that A(j) is the maximum
    global real xmax
    xmax ←A (1),  j ← 1
    for i ← 2 to n do
    if A (i) >  xmax then xmax ← A (i); j ← i;
 endif
repeat      //j,n, A(i:n) are formal parameters
end MAX // i is local parameter
```

Suppose n = 10
procedure MAX is called as follows
MAX (A, 10, j)

Input/ Output statements

read(argument list) ;

print (argument list);

Accepted rules of style for structured programs

- Every procedure should specify its input and output variables.
- The meaning of local variables should be defined.
- The flow of program should be forward except for loops.
- Indentation rules should be followed so that computational part can easily be identified.

Accepted rules of style for structured programs contd..

- Documentation should be short and meaningful.
- Use procedures (subroutines) where appropriate.
- For loop is preferred when the number of times to repeat is known.
- While loop is used when the condition is based on a value calculated inside.

RECURSION

- It is a powerful programming technique.
- Recursive programs may need more execution (than corresponding iterative ones) on some compilers.
- Direct Recursion- when a procedure includes a call to it self.
- Indirect Recursion- when a procedure calls another procedure which then causes the original procedure to be reinvoked

Examples of Recursive programs

1. Fibonacci sequence 0,1,1,2,3,5,8,13,21,34,…

   $F_0 = 0$, $F_1 = 1$, $F_i = F_{i-1} + F_{i-2}$, i.>1

   Procedure F(n)

   // return the nth Fibonacci number//

   integer n

   if n<=1 then return (n)

   else return (F(n-1) + F(n-2))

   endif

   end F

Examples of Recursive programs contd..

2. // Factorial of a number

// N! = N * (N-1)* (N-2)*…3* 2* 1

```
Procedure Fact(n)
  // returns the factorial of n//
  integer n
  if n<=1 then return 1
  else return n* Fact(n-1)
  endif
  end Fact
```

Examples of Recursive programs contd..

3. Euclid algorithm for GCD

```
procedure GCD (a,b)
        //Assume a > b>0
//If a <b then in the next step GCD (a,b) becomes GCD (b,a)
        If b=0 then return (a) //so the condition a>b reduces
        //the number of steps
        else return (GCD ( b,a mod b ))
end GCD
```

Exercises

- Write an algorithm to find maximum of any N numbers
- Write an algorithm to interchange any two numbers
- Write an algorithm to display smallest and next smallest of any three numbers
- Write an algorithm to display largest and next largest of any N numbers

Algorithm to interchange two numbers

```
// Interchange or swap two numbers
read x, y
 t = x
 x = y
 y = t
print x, y
```

Example: Consider 6 and 5, so x = 6 and y = 5

t = 6, x = 5, y = 6 so, x and y are interchanged

Algorithm to display smallest and second smallest of three distinct numbers

```
read a, b, c
 x =a    // if x < y < z then x will be smallest and
 y= b  // y will be second smallest
 z = c
 if x > y then t = x, x = y, y = t
 if y > z then t = y, y = z,  z = t
  if x > y then t = x, x = y, y = t
 print (“ first smallest is “, x)
 print (“ second smallest is “, y)
```

Example: smallest and second smallest of three distinct numbers

- Consider 8, 3 1

  x = 8, y = 3, z = 1

As x > y exchange so, x =3 and y = 8 now

As y > z exchange so, x = 3, y = 1 and z = 8

As x > y exchange so, x =1, y = 3 and z = 8

Hence smallest is x = 1 and

next smallest = y = 3

Algorithm to display largest and second largest in the given array

```
// read array A with n numbers
 read  n
for i = 1 to 10
  Read A(i)
 repeat
// store in A(1) largest and  A(2) next largest out of A(1) and A(2)
  if A(1) < A(2) {
   x = A(2), y = A(1)
  }
  else
 x = A(1), y = A(2)
// compare other numbers with these two  and replace large and second large
 for i = 3 to n
   if ( A(i) > y)
     if (A(i) > x) {
      y = x
      x = A (i)
  }
else
  y = A (i)
}
  print (" first Largest is ", x)
  print (" second Largest is ", y)
```

Example: Largest and next largest in an Array of N numbers

Consider 5 numbers -10, 10, 14, 12, 8

So, N = 5, A(1) = -10, A(2)=10, A(3)=14, A(4) =12, A(5)=8

Without loss of generality, assume, largest of A(1) and A(2) is x and smallest of A(1) and A(2) is y so, x is largest and y is next to largest; hence x = 10 and y = -10

The algorithm is similar to finding maximum in an array

Compare A(3), A(4) and A(5) with y and x and change x and y when needed

A(3) =14 > y= -10 and A(3) =14 > x =10 also, so, y=10, x= 14

A(4)= 12> y =10 but A(4) =12 < x=14, so, y=12 and x=14

A(5) = 8 < y=12 so x=14 and y = 12 // no change to x and y

Hence Largest is 14 and next largest is 12

Exercises

- Write algorithm to find the sum of digits of an integer.
- Write algorithm to test whether a giver number is prime or not.
- Write algorithm to compute m to the power n.
- Write algorithm to test whether a given year is a leap year or not.
- Write algorithm to reverse the digits of an n digit number.

Write algorithm to find the sum of digits of an integer

```
// sum of digits of an integer
  read N
   sum = 0
  while N < > 0
   rem = N mod 10
    sum = sum +rem
    N = N/10
    repeat
 print sum
```

Example: Let N = 123

123 < > 0

rem = 3, sum = 3, N = 12

12 < > 0

rem = 2, sum = 5, N = 1

1 < > 0

rem = 1, sum = 6, N = 0

Algorithm to test whether a giver number is prime or not

```
// To test whether given integer is prime or not
  read n
   for i=1 to n/2
    if ( n mod i) = 0 goto 10
     else
      repeat
      if (i = (n/2) +1) print (“ Number is prime”)
    else
   10:  print( “ Number not prime”)
```

Algorithm to compute m power n

```
// Computes m power n
// m power n  is m*m*…*m (n times)
 read m, n
 mpowern = 1
 for i = 1 to n
 mpowern = mpowern * m
 repeat
 print mpowern
```

Algorithm to check for leap year

```
// If year divisible by 4 and not divisible by 100 or
// if year divisible by 100 then it is also divisible by 400
// such an year is called a leap year
// read a four digit integer which represents an year
read year
if ((year mod 4 = 0) and (year mod100 < > 0)) or
((year mod 100 = 0) and (year mod 400 = 0)) then
print (“ Given year is a leap year”)
else print (”Given year is not a leap year)
```

Reverse the digits of an n digit number

```
// Display an n digit number in reverse order
read n, d
// n is the number and d is the number of digits
revn =0
for i = 1 to d
rem =n mod 10
revn =revn+rem*10 power d-i
print revn
```

Example: Let d =3 and n =265

for i = 1 to 3

i =1,rem=5, revn =5*10 *10=500

i=2, rem=6, revn=500+60=560

i=3, rem=2, revn=560+2=562

Number in reverse order 562

Questions and Answers on Problem Solving

1. What are the steps in problem solving on computer?

Steps in problem solving on computer

Answer

- Define the problem
- Analyze the solutions
- Develop the Algorithm
- Write the pseudocode or draw the flowchart
- Code using a programming language
- Debugging and Execution
- Verification and Testing

2. What is a pseudocode? How it is different from algorithm?

Answer

Algorithm is a step by step method to find the desired solution

Pseudocode is similar to the programming code in form except for the syntax of the program

Example: Compute the average of two numbers

Algorithm:

Read two numbers

Add the two numbers and store in sum

Divide the sum by 2 and store in average

print average

Pseudocode:

```
Read N1, N2
Sum = N1+ N1
average = sum/2
print Sum
```

3. What are the characteristics of algorithms?

Answer: Characteristics of algorithms

1. Each operation must be definite, i.e. it must be clear what should be done e.g. Compute 125/5, add 60 to 70

Example of not definite or ambiguous operations 5/0, add 2 or 3 to x.

2. Each operation should be effective i.e. it can be performed in a finite amount of time

3. Algorithm must terminate after a finite number of operations

4. How can we refer a number of variables of the same type?

Answer: A number of variables of the same type are referred to using Arrays

5. What is dimension and index in an array?

Answer:

Dimension signifies the maximum number of variables that can be stored in the array

Index is used to refer to a particular variable, for example, if A is an array and index is 3 means the third element is A(3)

6. What are the three important constructs used in any program?

Answer:

Sequence

Selection

Iteration and Repetition

7. When for loop is preferred and when while loop is preferred?

Answer:

For loop is preferred when the number of times to repeat is known.

While loop is used when the condition is based on a value calculated inside.